PYTHON PROGRAMMING

MODULE-2

MODULE-2

Strings: Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The stringformat method.

Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures.

Lists: List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list a$_2$nd range, Nested lists, Matrices.

ACompoundDataType:

- Thebuilt-intypesofdataare**int,float,bool,str,alongwithlistsandpaired data**.
- Strings,lists,andpaireddatadifferbecausetheyaremadeofsmallerelements(e.g., a string is made of characters).Such types are called **compound data types.**
- Theycanbetreatedeitherasasingleunitorasindividualparts,offeringuseful flexibility in programming.

Workingwithstringsasinglethings:

Astringisalsoanobject.Soeachstringinstancehasitsownattributesandmethods.

```
In [1]: our_string = "Hello, World!"
        all_caps = our_string.upper()
        all_caps

Out[1]: 'HELLO, WORLD!'

In [2]: all_lower=our_string.lower()
        all_lower

Out[2]: 'hello, world!'
```

Workingwithstringsasinglethings:

- The**upper()**methodconvertsallcharactersinastringtouppercase,creatinganewstring without changing the original.
- Otherusefulmethodsinclude:
  - **lower()**–convertsallcharacterstolowercase.
  - **capitalize()**–makesthefirstcharacteruppercase.
  - **swapcase()**–switchesuppercasetolowercaseandviceversa.
- Toexploremorestringmethods,youcan:ChecktheHelpdocumentation,or
- Simplytypedir(str)inaneditorlikejupyter,Thonny,Spydertoviewavailablemethods.

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings.

```
print(dir(str))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

Stringmethodsexample:
swapcase()

```
greet="Hello, World"
xx=greet.swapcase()
print(xx)
```

```
hELLO, wORLD
```

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings.

UsefulStringMethods

1) upper()-convertsalllowercasecharactersinastringtoupperecase
2) Lower()-ConvertsallUppercasechracteresinastringtoLowercase
3) Isupper()-Returnstrueifallcharactersofthestringareinuppercaseorelsereturnsfalse
4) Islower()-Returnstrueifallcharactersofthestringareinlowercaseorelsereturnsfalse
5) 3)and4)ReturnBooleanValues

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings. Useful

String Methods

- upper()
- lower()
- isupper()
- islower()

```
somestring="Oh! My God Python"
```

```
somestring
```

```
'Oh! My God Python'
```

```
somestring.upper()
```

```
'OH! MY GOD PYTHON'
```

```
somestring.lower()
```

```
'oh! my god python'
```

```
mystr="Hi Sharnga!"
mystr.islower()
```

```
False
```

```
mystr.isupper()
```

```
False
```

```
mystr1="HI"
mystr1.islower()
```

```
False
```

```
mystr1.isupper()
```

```
True
```

```
"KING".lower()
```

```
'king'
```

```
'king'.upper()
```

```
'KING'
```

```
'KING'.isupper()
```

```
True
```

```
"king".islower()
```

```
True
```

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings. The

isX() Methods – Return Boolean Values

- Isalpha()-ReturnsTrueifthestringconsistsonlyoflettersandisn’tblank
- isalnum()-ReturnsTrueifthestringconsistsonlyoflettersandnumbersandisnot blank
- Isdecimal()-ReturnsTrueifthestringconsistsonlyofnumericcharactersandisnot blank
- Isspace() - Returns True if the string consists only of spaces, tabs, and newlines and is not blank
- Istitle()-ReturnsTrueifthestringconsistsonlyofwordsthatbeginwithanuppercase letterfollowedbyonlylowercaseletters

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings. The

isX() Methods – Return Boolean Values

```
"Python".isalpha()
```
True

```
"Python3".isalnum()
```
True

```
"Python3.0".isalnum()
```
False

```
"Python".isalnum()
```
True

```
"180576".isdecimal()
```
True

```
''.isspace()
```
False

```
'  '.isspace()
```
True

```
'This Is The Title'.istitle()
```
True

```
'This is the title'.istitle()
```
False

```
'This IS The Title'.istitle()
```
False

```
"This Is The Title 1".istitle()
```
True

```
("Python" + "Programming").istitle()
```
False

```
("Python " + "Programming").istitle()
```
True

```
"P Y T H O N".istitle()
```
True

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings.

Thestartswith()andendswith() Methods

The startswith() and endswith() methods return True if the string value they are called on begins or ends (respectively) with the string passed to the method; otherwise, they return False.

```
'Python is run on Anaconda'.startswith('Python')
```

```
True
```

```
omg="Python is run on Anaconda"
```

```
omg.startswith("Python")
```

```
True
```

```
omg.endswith("conda")
```

```
True
```

```
omg.endswith("on Anaconda")
```

```
True
```

Workingwithstringsasinglethings:

There are more than 70 methods available to work on strings.

Thepartition()Method:

- **thepartition()methodisastringmethodusedto split a string into three parts based on a given separator.**
- **Usage:string.partition(separator)**
- **Parameters:separator→Thestring(substring)to search for in the main string.**
  - **Returns:Atupleofthreestrings:**
    - **Thepartbeforetheseparator**
  - **Theseparatoritself**
  - **Thepartafterthеseparator**

```
mystr="Hello World!"
mystr.partition('W')
```

```
('Hello ', 'W', 'orld!')
```

```
mystr.partition('World')
```

```
('Hello ', 'World', '!')
```


.

Workingwithstringsasinglethings:

Therearemorethan70methodsavailabletoworkonstrings. The string justification Methods :

- **ljust()→Returnsaleft-alignedstringofgivenwidth,paddingontheright.**
- **rjust()→Returnsaright-alignedstringofgivenwidth,paddingontheleft.**
- **center()→Returnsacenter-alignedstringofgivenwidth,paddingonbothsides.**

```
word="Python"
word.rjust(10)
```

```
'    Python'
```

```
word.center(10)
```

```
'  Python  '
```

```
word.ljust(10)
```

```
'Python    '
```

```
word.center(10,'*')
```

```
'**Python**'
```

Workingwiththepartsofastring:

Indexing:

- Theindexingoperator([])isusedtoaccessspecificcharactersinastring.
- Eachcharacterinastringhasapositionnumbercalledanindex.
- Indexingstartsfrom0andisfromleftto right.
- Negativeindexingstartsfrom-1canbeusedfromrighttoleft
- Ifindexisoutsidetherangepythonwillraiseanindexerror.
- Indexingreturnsastring

*enumerate*canbeusedtovisualizetoindex

```
list(enumerate(word))
```

```
[(0, 'b'), (1, 'a'), (2, 'n'), (3, 'a'), (4, 'n'), (5, 'a')]
```

Workingwiththepartsofastring:

Indexinga string:

Character	b	a	n	a	n	a
Index(+)	0	1	2	3	4	5
Index(–)	-6	-5	-4	-3	-2	-1

```
word="banana"
print(word[0])
print(word[2])
print(word[-1])
print(word[-3])
```

```
b
n
a
a
```

```
In [1]: word="banana"

In [2]: word[6]
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[2], line 1
----> 1 word[6]

IndexError: string index out of range
```



Index out of bound

Workingwiththepartsofastring:

- **ExamplesofIndexing-lists**

```
In [3]: oddnum=[1,3,5,7,9]

In [4]: oddnum[4] #index starts from 0
Out[4]: 9

In [5]: colours=["blue","orange","red","green","yellow"]

In [6]: colours[2]
Out[6]: 'red'
```

Lengthfunction`len()`:

- **Applyinglengthfunction,len(),toastringreturnsthenumberofcharactersina string**
- **Theindexvaries0to(lengthofthestring-1)**

```
In [3]: word="got you"
        len(word)

Out[3]: 7

In [4]: #to get the last letter of the string
        word[len(word)-1]

Out[4]: 'u'

In [8]: #to get the first of the string
        word[-(len(word))]

Out[8]: 'g'
```

Traversal and `for` loop:

- Traversal of a string means visiting each character in the string one by one.
- Strings are sequences of characters and can easily traversed using loops.
- Using while loop for traversal needs more lines of code and lacks clarity. Additionally `len()` function needs to be used to control the loop for termination.
- `for` loop can easily iterate over the elements in a string.

```
: #Traversal using while loop
word="orange"
i=0
while(i<len(word)):
    print(word[i],end=",")
    i+=1
```

```
o,r,a,n,g,e,
```

```
: #Traversal using for loop
word="orange"
for i in word:
    print(i,end=",")
```

```
o,r,a,n,g,e,
```

Traversalandforloop:

- Usingconcatenationandforlooptogenerateaabecedarianseries.
- Abecedarianreferstoaseriesorlistwhichhaveelementsinalphabeticalorder.

```
prefix="JKLMN"
suffix="awk"
for ch in prefix:
   print(ch+suffix)
```

```
Jawk
Kawk
Lawk
Mawk
Nawk
```

Slicesinstrings:

- Asubstringisapartofastring.
- Asubstringisobtainedbyslicingastring.
- Theoperatorforslicingis[startindex:endindex+1]
- The operator returns a substring starting from a character at start index to the character at end index.
- Thedefaultstartindexis0
- Thedefaultendindexis-1orlen()-1

```
phrase = "Slice is not a drink here"
print(phrase[0:5])
print(phrase[9:12])
print(phrase[:-1])
print(phrase[0:])
```

```
Slice
not
Slice is not a drink her
Slice is not a drink here
```

Slicing `list`:

- **Similar to a string a list could also be sliced as the elements of the list are identified by indexes in sequence.**
- **When a list is sliced using the slice operator [start index : end index – 1] , the elements from start index to end index are returned**

```
mylist=["red",'Raja','c',34,8.00,"/"]
```

```
mylist
```

```
['red', 'Raja', 'c', 34, 8.0, '/']
```

```
mylist[3:6]
```

```
[34, 8.0, '/']
```

Few moreexamplesofSlices:

- Inthesliceoperationiftheendindex is more than the len( ) of the string then the substring returned contains all characters fro start index till end.

- ThiswillalsonotresultinŌoutor rangeŌ error

```
phrase
```

```
'Slice is not a drink here'
```

```
phrase[-5:-2]
```

```
' he'
```

```
phrase[-3:-5]
```

```
''
```

```
phrase[4:56]
```

```
'e is not a drink here'
```

StringComparison:

StringComparisoninPythonistheprocessofcheckingwhethertwostringsareequal, unequal, or how they relate lexicographically (alphabetical order).

```
#string Comparison
word="Monkey"
if word=="Monkey":
   print("The strings match")
else :
   print("The strings do not match")
```

```
The strngs match
```

```
#Another way of comparison
word="Tiger"
if word  < "Lion":
    print(word,"comes before Lion")
else:
   print(word, "comes after Lion")
```

```
Tiger comes after Lion
```

Stringare immutable:

- Mutable(Changeable:Canbechangedaftercreation→e.g.,list,dict, set.
- Immutable(Unchangeable):Cannotbechangedaftercreation→e.g.,str,tuple,int.
- Onceastringiscreated,itcannotbemodifiedandhenceitisimmutable.
- Tryingtomodify,theoperationgivesanerror

```
: #Trying to change the first letter of the string
word="Python"
word[0]='M'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[7], line 3
      1 #Trying to change the first letter of the string
      2 word="Python"
----> 3 word[0]='M'

TypeError: 'str' object does not support item assignment
```

Stringare immutable:

- Tomakechangesinthestringtheestonecandoiscreateanewstringthatisa variation on the original.
- Thiscouldbeachievedwiththecombinationofslicingand concatenation.

```
#Trying to change the first letter
#using slice and concatenation
word="Python"
newword = 'M'+ word[1:]
newword
```

```
'Mython'
```

`in` and `not in` operators:

- `in` operator: Returns True if a substring is found within a string.
- `not in` operator: Returns True if a substring is not found within a string.

```
"name" in "my name is Ranna"
```

```
True
```

```
"name" not in "my name is Ranna"
```

```
False
```

```
"Anna" in "my name is Ranna"
```

```
False
```

```
"Anna" in "my name is RAnna"
```

```
True
```

```
"" in "my name is RAnna"
```

```
True
```

```
" " in "my name is RAnna"
```

```
True
```

```
"y n" in "my name is Ranna"
```

```
True
```

```
"is " not in "my name is Ranna"
```

```
False
```

in and not in operators:

Thonny IDE

- Programtoremovevowelsfromastring

```
1  vowels="aeiouAEIOU"
2  mystring=input("Enter a string to remove vowels : ")
3  strnovowel=""
4  for i in mystring:
5      if i not in vowels :
6          strnovowel = strnovowel+i
7  print("The string after the removal of vowels : ", strnovowel)
```

Output:

```
Enter a string to remove vowels : Anaconda is bigger than Python
The string after the removal of vowels :  ncnd s bggr thn Pythn
```

A `find( )` function:

Description:

- The `find()` method in Python is used to search for a substring within a string.
- It returns the index (position) of the first occurrence of the substring.
- If the substring is not found, it returns -1.

Syntax:

```
string.find(substring,start,end)
```

- substring → The string to be searched.
- start (optional) → The position to start the search.
- end (optional) → The position to end the search.

`Afind()` function: Example :

```
text = "Python programming is fun"
# Find the position of the word 'programming'
pos = text.find("programming")
print("Position:", pos)
pos = text.find("in",14)
print("Position of in :", pos)
pos = text.find("is",19,23)
print("Position of is:", pos)
pos = text.find("fun",19,23)
print("Position of fun:", pos)
```

Output:

```
Position: 7
Position of in : 15
Position of is: 19
Position of fun: -1
```

A `find( )` function:

User defined `my_find()` function

```
1  def my_find(string,needle):
2      for index, letter in enumerate(string):
3          if letter == needle:
4              return index
5      return -1
6
7  mystring="Python is fun!"
8  print(mystring.find('o'))
9  print(my_find(mystring,'o'))
```

Output:

```
>>> %Run -c $EDIT
4
4
```

Loopingand Counting:

Programtocountthenumberoftimes(Frequency)acharacterappearsina string.

```
1  def count_a(text):
2      count = 0
3      for letter in text:
4          if letter == "a":
5              count += 1
6      return(count)
7  print(count_a("banana") == 3)
```

```
>>> %Run -c $EDITOR_CONTENT
  True
```

OptionalParameters:

To find the locations of the further occurrence of a character in a string one can modify the find function by adding a third parameter (start) for the starting position in the search string.

```
1  def my_find_1(string,needle,start):
2      for index, letter in enumerate(string[start:]):
3           if letter == needle:
4               return index+start
5      return -1
6
7  mystring="Python is fun!"
8  print(my_find_1(mystring,'o',5))
9  #output is -1 because after index 5 there is no o
10
```

```
>>> %Run findfunstart.py
  -1
```

OptionalParameters:

- **my_findandmy_find_1couldbecombinedwithoptionalparameterstart=0**
- **In this case if the third argument is provided in the function call it will be consideredand if the third argument is left outthen the program considers the default value start=0.**

```
1  def my_find_2(string,needle,start=0):
2      for index, letter in enumerate(string[start:]):
3          if letter == needle:
4              return index+start
5      return -1
6
7  mystring="Python is fun!"
8  print(my_find_2(mystring,'o',5))
9  print(my_find_2(mystring,'o'))
```

```
>>> %Run findfunstart.py
-1
4
```

- Thecodeintheline8callsthefunctionwiththethirdparameterstartpassingavalue5
- Thecodeintheline9callsthefunctionwiththethirdparameterstartwithdefaultvalue0

OptionalParameters:

- Adding another optional parameter to find makes it search from a starting position, upto but not including the end position

```
1  def my_find_2(string,needle,start=0,end=-1):
2      for index, letter in enumerate(string[start:end]):
3          if letter == needle:
4              return index+start
5      return -1
6
7  mystring="Python is fun!"
8  print(my_find_2(mystring,'o',5))
9  print(my_find_2(mystring,'o'))
```

Builtin`find()`Method:

- Builtfindmethodcanfindeventhesubstringsinadditionsinglecharacter.
- Itfindsthefirstoccurrenceofasubstringorcharacterandreturnsrespectiveindex.

```
"banana".find("a")
```

```
1
```

```
"banana".find("nan")
```

```
2
```

```
"banana".find("na",3)
```

```
4
```

Cleaningupyour strings:

AdditionalInformation:

Thepreviouscodesimplified

```
1  import string
2
3  def remove_punctuation(text):
4      return ''.join(ch for ch in text if ch not in string.punctuation)
5
6  my_story = "hi! How are you?. Hope you are doing fine."
7  print(remove_punctuation(my_story))
```

```
hi How are you Hope you are doing fine
```

Thestringformat method:

- Theformat()MethodinPython(Description)
- Theformat()methodisusedtoinsertvaluesintostringsatspecificplaceholderpositions.
- Placeholdersarewritteninsidecurlybraces{}withinthestring.
- Ithelpsincreatingdynamicandreadablestringswithoutusingconcatenation.
- Youcanusepositional,keyword,orindexedargumentsinside{}.
- Generalsyntax:**`string.format(value1,value2,...)`**

```
1 name = "Alice"
2 age = 25
3 print("My name is {} and I am {} years old.".format(name, age))
```

```
My name is Alice and I am 25 years old.
```

Thestringformat method:

Fewmoreexamples:

```
1 phrase = "His name is {0}!".format("Arthur")
2 print(phrase)
3 name = "Alice"
4 age = 10
5 phrase = "I am {1} and I am {0} years old.".format(age, name)
6 print(phrase)
7 phrase = "I am {0} and I am {1} years old.".format(age, name)
8 print(phrase)
```

```
His name is Arthur!
I am Alice and I am 10 years old.
I am 10 and I am Alice years old.
```

```
1 x = 4
2 y = 5
3 phrase = "2**10 = {0} and {1} * {2} = {3:f}".format(2**10, x, y, x * y)
4 print(phrase)
```

```
2**10 = 1024 and 4 * 5 = 20.000000
```

Thestringformat method:

Fewmoreexamples:

```
1 letter = """
2 Dear {0} {2}.
3 {0}, I have an interesting money-making proposition for you!
4 If you deposit $10 million into my bank account, I can
5 double your money ...
6 """
7 print(letter.format("Paris", "Whitney", "Hilton"))
8 print(letter.format("Bill", "Henry", "Gates"))
```

```
Dear Paris Hilton.
Paris, I have an interesting money-making proposition for you!
If you deposit $10 million into my bank account, I can
double your money ...


Dear Bill Gates.
Bill, I have an interesting money-making proposition for you!
If you deposit $10 million into my bank account, I can
double your money ...
```

Tuplesareusedforgroupingdata

Whatisa tuple?:

- Atupleisanimmutablesequencetype:oncecreated,itselementscannotbechanged(unlikelists).
- It isusefulforgroupingtogetherrelatedvalues,whereyouintendthatthegroupingisfixed(thatis, the structure is not going to be mutated). For example: a person's first name, last name, birth-year; or the (x,y) coordinates of a point.
- Syntax:acomma-separatedlistofvalues,typically(butoptionally)enclosedinparentheses.
- `t=("Alice",30,"Engineer")`
- Creating a tuple with a single element requires a trailing comma, otherwise Python interprets the parentheses as just grouping. Example:
  - `single=(5,)  #thisisatupleoflength1`
  - `not_a_tuple=(5)  #thisisjusttheinteger5`

Tuples

Tuplesareusedforgroupingdata

Basicoperations&syntax:

- **Literalcreation:t=(v1,v2,v3)**
- Indexing:Onecanaccesselementslikeyoudoforlistsorstrings:**t[0]**
- **Slicing:**Onecanslicetuplesandgetanewtuple:**t[0:2]**
- **Concatenation:**
  - onecanformanewtuplebyconcatenatingtwotuples:**t3=t+(new_val1,new_val2)**
- **Immutability:**Onecannotdo**t[0]**=something—thiswillraiseaTypeError

Tuplesareusedforgroupingdata

Example:groupingdata:

```
1 #Tuples cannot be mutated
2 numbers=(18,21,13,54,65,26,77)
3 print(numbers[0])
4 print(numbers[3:6])
5 print(numbers[:-1])
6 #Tuples can be changed by Slice + Concatenate
7 numbers = numbers[:5]+(34,56,89)+numbers[6:]
8 print(numbers)
9 #Thus tuple cannot be mutated but It can be built newly
```

```
18
(54, 65, 26)
(18, 21, 13, 54, 65, 26)
(18, 21, 13, 54, 65, 34, 56, 89, 77)
```

TuplesAssignment

Whatistupleassignment(unpacking)?

- Tuple assignmentisa feature in Python that allows you to assign multiple variables atoncebyunpacking a tuple of values into a tuple of variables.
  - **Thesyntax:**
    - `(var1,var2,var3,…)=(val1,val2,val3,…)`
- Thenumberofvariablesontheleftmustmatchthenumberofvaluesonthe right.
- Alltheexpressionsontherightsideareevaluatedbeforeanyassignmenthappens;thusyou can use this to swap two variables in one line.

```
1 x,y,z=(2,3,4)
2 print(f"The value of x is {x} y is {y} and z is {z}")
```

```
The value of x is 2 y is 3 and z is 4
```

TuplesAssignment

Whatistupleassignment(unpacking)?

- Tuple assignmentisa feature in Python that allows you to assign multiple variables atoncebyunpacking a tuple of values into a tuple of variables.
  - **Thesyntax:**
    - `(var1,var2,var3,…)=(val1,val2,val3,…)`
- Thenumberofvariablesontheleftmustmatchthenumberofvaluesonthe right.
- Alltheexpressionsontherightsideareevaluatedbeforeanyassignmenthappens.Example:

```
1 x,y,z=(2,3,4)
2 print(f"The value of x is {x} y is {y} and z is {z}")
```

```
The value of x is 2 y is 3 and z is 4
```

TuplesAssignment

Fewmore examples

```
1  t = ("Alice", 30, "Engineer")
2  (name, age, profession) = t
3  print(name)          # "Alice"
4  print(age)           # 30
5  print(profession)    # "Engineer"
```

```
Alice
30
Engineer
```

Pairs

```
1  pairs = [(1,2), (3,4), (5,6)]
2  for (x, y) in pairs:
3      print(x, y)
```

```
1 2
3 4
5 6
```

Swapping

```
1  #swapping
2  x,y={3,4}
3  print(f"Before swap x = {x} and y = {y}")
4  x,y=y,x
5  print(f"After swap x = {x} and y = {y}")
```

```
Before swap x = 3 and y = 4
After swap x = 4 and y = 3
```

Tuplesasreturnvalues

- InPython,afunctioncanreturnonlyoneobject.
- However,thatsingleobjectcanbeatuple,allowingustogroupandreturnmultiple values together.
- Thismakesitpossibletoreturnrelatedresultsinonecall,improvingclarityandreducing extra variables.

Keyidea:"Onereturn,manyvalues—groupedinatuple." Examples of practical uses:

- Returningabatsman'shighestandlowestscores.
- Returningmeanandstandarddeviationofadataset.
- Returning(year,month,day)fromadatecomputation.
- Returning(rabbits,wolves)populationcountsinanecologicalmodel.

Tuplesasreturnvalues

ExampleProgram

```
1  def divide(a, b):
2      quotient = a // b
3      remainder = a % b
4      return quotient, remainder  # returns a tuple
5  # Using the returned tuple
6  q, r = divide(20,6)
7  print(f"Quotient is {q} and Reminder is {r}")
8
```

```
Quotient is 3 and Reminder is 2
```

ComposabilityofDataStructures

Meaning:

- **Composabilitymeanscombiningsimpledatastructures(liketuples,lists)tobuildmorecomplex ones.**
  - **Onecanneststructuressuchastuplesinsidelists,listsinsidetuples,ortuplesoftuples.**
- **Purpose:**
  - **Modelreal-worlddata(e.g.,astudentrecordorcoordinateset).**
  - **Grouprelatedvaluesintomeaningful,structuredunits.**
  - **Passorreturncomplexdataeasilyasasingleobject.**
- **Syntaxexamples:**

```
1 point1 = (2, 3)
2 point2 = (5, 7)
3 triangle = (point1, point2, (4, 8))# tuple of tuples
4 students = [("Alice", [88, 92]), ("Bob", [75, 81])]
```

ListandListValues:

- **A`list`inPythonisasequenceofvalues(elements)inwhicheachelementcanbeofanytype,and different elements in the same list can be different types.**
- **Listsareordered(eachelementhasaposition/index)andmutable(youcanchangethecontents).**
- **Basicsyntaxforalistliteral:`list_name=[item1,item2,item3,…]`**
- **Emptylistisrepresentedby`list_name=[]`**
- **Alistwithinalistiscallednestedlist.**
- **Example:**

```
1 my_list = [10, 20, 30, 40]
2 other_list = ["spam", "bungee", "swallow"]
3 mixed = ["hello", 2.0, 5, [10,20]]
```

MeritsandOperationsonlists:

- **Whenyouhaveacollectionofvaluesyouwanttostore,referto,iterateover,mutate,sort,etc.,alist is a natural choice.**
- **Becauselistscancontainmixedtypes(thoughforclarityyouoftenkeeptypesconsistent)theyare very flexible.**
- **Becausetheyaremutable,youcaninsert,delete,replaceelements—usefulfordynamiccollections.**
- **Listssupportindexing,slicing,concatenation(+),repetition(*),membershiptesting(in,notin), length retrieval (len()), and more.**

```
1 grades = [88, 92, 79, 85]
2 names = ["Alice", "Bob", "Cathy"]
3 mixed = ["Alice", 19, 3.15, grades]
4 print(mixed)
5 print(mixed[3])
```

```
['Alice', 19, 3.15, [88, 92, 79, 85]]
[88, 92, 79, 85]
```

Accessingelements:

- **Indexing**
  - Youcanaccessanelementofalistbyusingthebracketoperator[]withanintegerindex.The first element has index 0.
  - Negativeindicesareallowed:numbers[-1]givesthelastelement,numbers[-2]thesecond-last.

```
1 grades
```

```
[88, 92, 79, 85]
```

```
1 names
```

```
['Alice', 'Bob', 'Cathy']
```

```
1 grades[2]
```

```
79
```

```
1 names[-2]
```

```
'Bob'
```

Accessingelements:

- **ListTraversal(iteratingviaindices)**
  - Visitingallelementsoritemsofalistiterativelyiscalledtraversal
  - **`While`**or**`For`**loopscouldbeusedfortraversing lists.

```
1 grades = [88, 92, 79, 85]
2 i = 0
3 while i < len(grades):
4     print(grades[i],end=" ")
5     i = i + 1
```

```
88 92 79 85
```

```
1 grades = [88, 92, 79, 85]
2 for item in grades:
3     print(item,end=" ")
```

```
88 92 79 85
```

Accessingelements:

- **ChangingandSlicingListElements**
  - Sincelistsaremutable,youcanassigntoanindexedposition
  - Onecanalsouseslicingassignmenttoreplacemultipleelements:

```
1 fruit = ["banana", "apple", "quince"]
2 fruit[0] = "pear"
3 fruit[-1] = "orange"
4 print(fruit)              #
```

```
['pear', 'apple', 'orange']
```

```
1 mylist = ['a','b','c','d','e','f']
2 mylist[1:3] = ['x','y']
3 mylist
```

```
['a', 'x', 'y', 'd', 'e', 'f']
```

Lengthofthelist(`len()`):

- **Thefunction`len`returnsthelengthofalist,whichisequaltothenumberofitselements.**
- **Examplesforthelengthoflistandnestedlist**

```
1 mylist = ['a','b','c','d','e','f']
2 len(mylist)
```

6

```
1 nested_list = ['spam!', 1, ['Hi','bye'], [1,2,3]]
2 print(len(nested_list))
```

4

Usageoflen()for traversing the list

```
1 mylist = ['a','b','c','d','e','f']
2 for i in range(len(mylist))
3     print(mylist[i],end= " ")
```

Listmembership

- **Using`in`and `not in`operators**
- **Theoperator`in`returns`True`ifanelementisinthelist, `False`otherwise.Similarly `not in`tests non-membership.**

```
1 horsemen = ['war', 'famine', 'pestilence', 'death']
2 print('pestilence' in horsemen)
3 print('debauchery' in horsemen)
4 print('debauchery' not in horsemen)
5 print('death' not in horsemen)
```

```
True
False
True
False
```

Listmembership

- **Quicktestsifavalueoccursinalist:helpfulformembershipchecking,errorchecking, filtering, etc.**
- **Becauselistscanbelarge,inprovidesasimple,readablecheckratherthanmanually looping through.**
- **Usageof`in`inthe`for`loopsfornestedlists**

```
1 student=[('Kiran',[21,45,67]),('Nandan',[23,45])]
2 for name,marks in student:
3     print(name)
4     for marks in marks:
5         print(marks,end=" ")
6     print()
```

```
Kiran
21 45 67
Nandan
23 45
```

Listoperations

- **Concatenation(Joining)oftwolistscouldbeachievedusingtheoperator‘+’calledthe concatenation operator**
- **Replication(Repetition)oflistelementscouldbeachievedusingtheoperator‘*”called the replication operator.**

Two lists name and age are concatenated using ‘+” Operator. The resulting new list contains elements of both the lists in sequence.

```
1 name=["John","Bolt"]
2 age=[34,55]
3 newlist = name + age
4 print(newlist)
```

```
['John', 'Bolt', 34, 55]
```

The list patriot is replicated 3 times using the * operator.

```
1 patriot=["My","India"]
2 patriot * 3
```

```
['My', 'India', 'My', 'India', 'My', 'India']
```

ListSlices

- Slicingalistisobtainingasublistofthelistusingsliceoperator `[start:end+1]`.
- Defaultvalueof `start` is0and `end+1` defaultstotheendofthe list.
- Syntax:list_name[start:end+1]

```
1 numbers = [10, 20, 30, 40, 50, 60]
2 print(numbers[1:4])
3 print(numbers[:3])
4 print(numbers[3:])
5 print(numbers[:])
```

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 20, 30, 40, 50, 60]
```

Listsare Mutable:

- **Unlikestrings,listsaremutable.Hencethelistelementscanbechangedthrough indexing.**

Indexassignment

```
1 fruit = ["banana", "apple", "quince"]
2 fruit[0] = "pear"
3 fruit[-1] = "orange"
4 print(fruit)              #
```

```
['pear', 'apple', 'orange']
```

Sliceassignment

```
1 mylist = ['a','b','c','d','e','f']
2 mylist[1:3] = ['x','y']
3 mylist
```

```
['a', 'x', 'y', 'd', 'e', 'f']
```

ListDeletion:

- **Listdeletionisdeletingthelistelementsusingindexorslices.**
- **Elementsofalistcouldbedeletedusing`del`statement.**
- **Syntax:`dellist_name[index],del_list_name[start:end+1]`**

```
1 a = ["one", "two", "three"]
2 del a[1]
3 a
```

```
['one', 'three']
```

```
1 a_list = ["a", "b", "c", "d", "e", "f"]
2 del a_list[1:5]
3 a_list
```

```
['a', 'f']
```

ObjectsandReferences:

- WhenavariableiscreatedinPython,itdoesnotdirectlystoretheobject'svalue;instead, it stores a reference (or address) to the object in memory.
- Anobjectisapieceofdatastoredsomewhereinmemory,andareferenceislikeatagor name that points to it.
- Forexample,whenyouwritea=[1,2,3],thelist[1,2,3]isstoredsomewhereinmemory, and the variable 'a 'refers to that list.
- Ifanothervariableisassignedto'a'(likeb=a),bothaandbrefertothesamelist—soa change made through one variable affects the other.
- Immutableobjects(likeintegersandstrings),ontheotherhand,createnewobjectsin memory when modified.
- Thus,objectsandreferencesexplainhowPythonmanagesvariables,memory,anddata sharing efficiently.

ObjectsandReferences (examples):

```
1 a="guava"
2 b="guava"
3 a is b
```

```
True
```

Object

isoperatorChecks whether two names refer to the same object

```
1 a=[1,2,3]
2 b=[1,2,3]
3 a==b
```

```
True
```

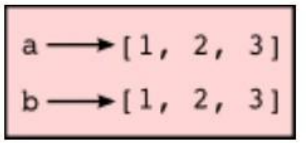
aand b have the same valuebut do not refer to thesame object.

```
1 a is b
```

```
False
```



Statesnapshot




```
1 id(a)
```

```
139061895539872
```

Reference

```
1 id(a)
```

```
139061895620608
```

Onlyfor understanding

```
1 id(b)
```

```
139061895539872
```

```
1 id(b)
```

```
139061851734400
```

Aliasing:

- Since variables refer to objects, if we assign one variable to another, both variables refer to the same object

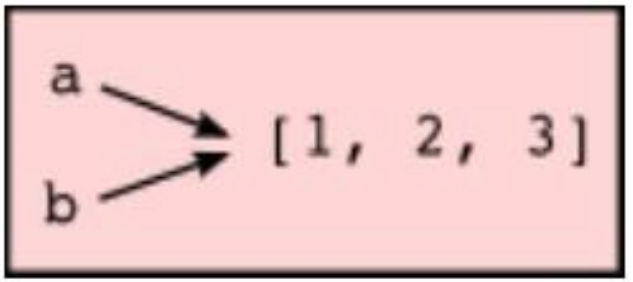
```
1  a=[1,2,3]
2  b=a
3  a is b
```

```
True
```

```
1  b[1]=99
2  a
```

```
[1, 99, 3]
```

StateSnapshot



Because the same list has two different names, aand b, we say that it is aliased. Changes made with one alias affect the other

aandbareAliased

CloningLists:

- **If we want to modify a list and also keep a copy of the original, we need to be able to make a copy of the list itself, not just the reference.**
- **This process is sometimescalled cloning, to avoid the ambiguity of the word copy.**

```
1 a=[1,2,3]
2 b=a[:]
3 b
```

```
[1, 2, 3]
```

```
1 b[0]=5
```

```
1 b
```

```
[5, 2, 3]
```

```
1 a
```

```
[1, 2, 3]
```

```
1 id(a)
```

```
139061594610048
```

```
1 id(b)
```

```
139061851700032
```

Justtoknowreferencesare different and objects are different

ListsandforLoop:

- **for**loopisusedtoiterateoverthelistsfortraversing.
- Thegeneralsyntaxis:

```
for <VARIABLE> in <LIST>:
    <BODY>
```

Example1

```
1 friends = ["Joe", "Zoe", "Brad", "Angelina", "Zuki"]
2 for friend in friends:
3     print(friend)
```

```
Joe
Zoe
Brad
Angelina
Zuki
```

Example2

```
1 for (i, v) in enumerate(["banana", "apple", "pear", "lemon"]):
2     print(i, v)
```

```
0 banana
1 apple
2 pear
3 lemon
```

ListParameters

- Passingalistasanargumentpassesitsreference,nota copy.
- Thismeansboththecallerandthefunctionrefertothesamelistobject(an alias).
- Example:

```
1  def double(lst):
2      for i in range(len(lst)):
3          lst[i] *= 2
4  num=[1,2,3,4,5]
5  double(num)
6  print(num)
```

```
[2, 4, 6, 8, 10]
```

ListMethods

- The**.(dot)**operatorcanalsobeusedtoaccessbuilt-inmethodsoflist objects.

```
1 mylist=[2,'x','y',4,'z']
2 mylist.insert(4,"m")
3 mylist
```

```
[2, 'x', 'y', 4, 'm', 'z']
```

`mylist.insert(4,'m'),`

`insert()` isamethodwhichoperatesonalist **`mylist`**andinsertsand element **`'m'`** at index **`4`**.

ListMethods

- **append()** isamethod whichinsertsorappendsanelementattheendofthe string

```
1 mylist=[1,4,5,7]
2 mylist.append('67')
3 mylist
```

```
[1, 4, 5, 7, '67']
```

- **Insert()** isalistmethodtakestwoparametersandinsertsagivenelementatthe given index

```
1 mylist=[1,4,5,7]
2 mylist.insert(2,67)
3 mylist
```

```
[1, 4, 67, 5, 7]
```

ListMethods

- **count()**methodcountsthenumberoftimesthegivenelementappearsinthe list.

```
1 mylist=[3,4,7,4,8]
2 mylist.count(4)
```

```
2
```

- **extend()**putsthepassedlistatendofthegiven list.

```
1 mylist=[4,5,6,7]
2 mylist.extend([3,2,1])
3 mylist
```

```
[4, 5, 6, 7, 3, 2, 1]
```

ListMethods

- **reverse()**reversestheorderofelementsinthelist

```
1 mylist=[1,2,3,4]
2 mylist.reverse()
3 mylist
```

```
[4, 3, 2, 1]
```

- **sort()**arrangestheelementsofthelistinorder (Sorting)

```
1 mylist=[7,2,9,5]
2 mylist.sort()
3 mylist
```

```
[2, 5, 7, 9]
```

ListMethods

- **index()**returnstheindexofanelementinthelist.

```
1 mylist=[4,5,2,7]
2 mylist.index(2)
```

```
2
```

- **remove()**removesthegivenelementfromthe list.

```
1 mylist=[4,5,2,7,24]
2 mylist.remove(2)
3 mylist
```

```
[4, 5, 7, 24]
```

ListMethods

- **pop()**methodinPythonisused toremoveandreturnanelementfroma list..

```
1 mylist=[4,5,2,7,24]
2 x=mylist.pop(3)
3 print(mylist,'and',x)
```

```
[4, 5, 2, 24] and 7
```

PureFunctionsandModifiers

Pure Functions:

- Functionsthatdonotmodifytheoriginaldataarecalled**PureFunctions.**
- Theyreturnanewvaluebasedontheinput.
- Safe to use without side effects.(The change in the list argument passed to afunction during the execution is called side effect)

```
1  def doubled(lst):
2      result = []
3      for x in lst:
4          result.append(x * 2)
5      return result
6
7  nums = [1, 2, 3]
8  print(doubled(nums))  # [2, 4, 
9  print(nums)
```

```
[2, 4, 6]
[1, 2, 3]
```

PureFunctionsandModifiers

Modifiers:

- Functionswhichtakelistsasargumentsandchangethemduringexecutionarecalled **Modifiers**.
- Changetheoriginalobjectdirectlyand Donotusuallyreturnanewlist.
- TheChangesintheargumentsarecalled**sideeffects**

```
1 def double_in_place(lst):
2     for i in range(len(lst)):
3         lst[i] *= 2
4
5 nums = [1, 2, 3]
6 double_in_place(nums)
7 print(nums)  # [2, 4, 6] (modified)
```

```
[2, 4, 6]
```

Functionsthatproducelists

- Functionsthatproducelistseitherreturnnewlists(likelist(),comprehensions,orcustomfunctions) or convert other iterable into lists.

```
1  def doubled(lst):
2      result = []
3      for x in lst:
4          result.append(x * 2)
5      return result
6
7  nums = [1, 2, 3]
8  print(doubled(nums))  # [2, 4, 
9  print(nums)
```

```
[2, 4, 6]
[1, 2, 3]
```

StringsandLists

- **Refermethodsjoin()andsplit()**

```
1 name="suresh raj"
2 namelist=name.split()
3 namelist
```

```
['suresh', 'raj']
```

```
1 name=["suresh", "raj"]
2 " ".join(name)
```

```
'suresh raj'
```

The `join( )` method:

- The join() method is useful when you have a list of strings that need to be joined together into a single string value.
- The join() method is called on a string, gets passed a list of strings, and returns a string.
- The returned string is the concatenation of each string in the passed-in list.

```
mylist=["I","am","confident","of","Programming","in Python"]
```

```
mylist
```

```
['I', 'am', 'confident', 'of', 'Programming', 'in Python']
```

```
" ".join(mylist)
```

```
'I am confident of Programming in Python'
```

```
",".join(["cat","rat","bat"])
```

```
'cat,rat,bat'
```

```
"XYZ".join(mylist)
```

```
'IXYZamXYZconfidentXYZofXYZProgrammingXYZin Python'
```

The `split()` method:

- The split() method does the opposite.
- It's called on a string value and returns a list of strings.
- The string is split wherever whitespace characters, the space, tab, or newline,are found.
- The whitespace characters are not included in the strings in the returned list.
- A delimiter string to the split() methodtospecifyadifferent stringtosplitupon.

```
"XYZ".join(mylist)
```

```
'IXYZamXYZconfidentXYZofXYZProgrammingXYZin Python'
```

```
'I am confident of Programming in Python'.split()
```

```
['I', 'am', 'confident', 'of', 'Programming', 'in', 'Python']
```

```
'IXYZamXYZconfidentXYZofXYZProgrammingXYZin Python'.split('XYZ')
```

```
['I', 'am', 'confident', 'of', 'Programming', 'in Python']
```

```
string="Oh! It's Easy! \n Let me try."
string.split('\n')
```

```
["Oh! It's Easy! ", ' Let me try.']
```

Cleaningupyour strings:

- Whenworkingwithstrings,weoftenencounterpunctuation,tabs,ornewlinecharacters—especially when reading text from files or the Internet.
- Fortaskslikecountingwordfrequenciesorcheckingspelling,it’sbettertoremovetheseunwanted characters.
- Sincestringsareimmutable,wecan’tmodifythemdirectly.
- Instead,wetraversetheoriginalstringandbuildanewonethatexcludespunctuation.

```
1  import string
2  #User defined function remove punctuations
3  def remove_punctuation(phrase):
4      phrase_sans_punct = ""
5      for letter in phrase:
6          if letter not in string.punctuation:
7              phrase_sans_punct += letter
8      return phrase_sans_punct
9  #main program
10 my_story="hi! How are you?. Hope you are doing fine."
11 print(remove_punctuation(my_story))
```

```
hi How are you Hope you are doing fine
```

listandrange

- Pythonhasabuilt-intypeconversion functioncalled**`list( )`**that tries to turn whatever you give it into a list.

```
1 name="suresh"
2 letters=list(name)
3 letters
```

```
['s', 'u', 'r', 'e', 's', 'h']
```

- **`range`**generates values lazily, meaningitproducesthemondemand instead of all at once.
- Althoughrange(1000000)represents a million numbers,
- Pythondoesn’tcreatethemallatonce —itgenerateseachvalueonlywhen needed.
- Here,theloopstopsati==5,soonly the first six values are produced.

```
1 x=[i for i in range(10) if i==3 ]
2 x
```

```
[3]
```

```
1 x=[i for i in range (10)]
2 x
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Nestedlists

- **Anestedlistisalistthatappearsasanelementinanother list.**

```
1 student=[('Kiran',[21,45,67]),('Nandan',[23,45])]
2 for name,marks in student:
3     print(name)
4     for marks in marks:
5         print(marks,end=" ")
6     print()
```

```
Kiran
21 45 67
Nandan
23 45
```

Matrices

- **Nestedlistsareoftenusedtorepresent matrices.**
- **Eachsublistrepresentsarowofmatrixelements.**

Matrix $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

```
1  mat=[[1,2,3],[4,5,6],[7,8,9]]
2  mat[1]
```

```
[4, 5, 6]
```

```
1  mat[1][2]
```

```
6
```

```
1  for i in range(3):
2      for j in range(3):
3          print(mat[i][j],end=" ")
4      print()
```

```
1 2 3
4 5 6
7 8 9
```

End of Module 2